

Writing clean and fast code in Julia

Lucas Bex Matteo Rossini Jeroen Tant

6 May 2026



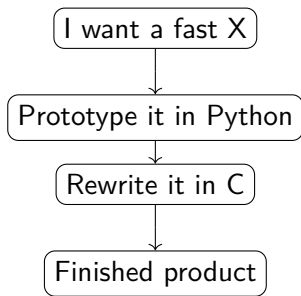
If you haven't downloaded materials, do that now

- ▶ Julia **1.12**: <https://julialang.org/downloads/>
- ▶ Workshop materials: <https://github.com/Electa-Git/JuliaWorkshop2026>
- ▶ (optional) VSCode Julia extension
- ▶ Header points to subdirectories in the workshop material

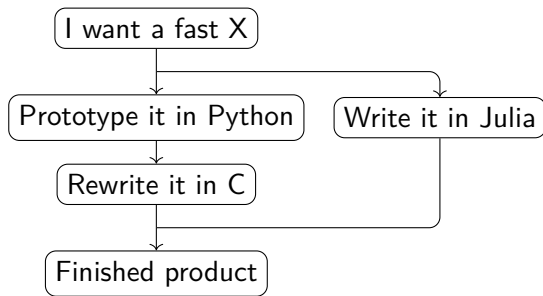
Outline

- ▶ 5' Introduction
- ▶ 35' Packages and environments
- ▶ 50' Performance optimization + model description and baseline script
- ▶ 55' Clean code
- ▶ 5' Closing remarks

Julia in a nutshell



Julia in a nutshell



- ▶ Just-in-time compiled
- ▶ Multi-paradigm
- ▶ Rich type system
- ▶ Multiple dispatch
- ▶ Easy to use



A toy project in Julia

How long will it take to get my paper published?

Let's build a *Monte Carlo simulation*.

Sources of uncertainty:

1. Time to *write* the paper: estimated to 60 days
 - ▶ Erlang distribution; shape = 60, scale = 1
2. Duration of the *peer review process*: average 90 days according to the journal
 - ▶ Exponential distribution; scale = 90



Environments

An environment determines what `import X` and `using X` mean in various code contexts and what files these statements cause to be loaded.

- ▶ Environments are managed from the *package mode* of the REPL
 - ▶ Type `]` to enter and Backspace to exit
 - ▶ Commands: `activate`, `status`, `add`, `develop...` Type `?` for help
- ▶ *Project environment*: a directory containing a `Project.toml` file

References:

- ▶ Pkg Documentation > [3. Managing Packages](#)
- ▶ Pkg Documentation > [4. Working with Environments](#)



What is there in my environment stack?

Julia environments are stacked

- ▶ Stack defined by `LOAD_PATH`; paths returned by `Base.load_path()`
- ▶ Usually: 1. `@` (active environment); 2. `@v#.#` (default environment); 3. `@stdlib`
- ▶ Stack composed bottom-up $\Rightarrow$ The topmost environment is faithfully embedded

Good practices:

- ▶ Create a separate environment for each task you happen to work on
- ▶ Keep your *default environment* (`@v#.#`) lightweight
 - ▶ Only tools for development
 - ▶ Don't use it as a "catch-all"

References:

- ▶ Julia Documentation > Manual > [Code Loading](#)



Packages

- ▶ Packages are for *reusing code* (by you or others)
- ▶ A package provides a module $\Rightarrow$ Defines a *namespace*
- ▶ Package versioning follows *Semantic Versioning*: <https://semver.org>
 - ▶ Version numbers follow the MAJOR.MINOR.PATCH format – and they do have a meaning!
- ▶ A package declares *dependencies* (the external packages it relies on)
 - ▶ If you develop a package, be sure to understand [version specifiers](#)

References:

- ▶ Pkg Documentation > [5. Creating Packages](#)
- ▶ [Semantic Versioning](#)
- ▶ Pkg Documentation > [7. Compatibility](#)
- ▶ Pkg Documentation > [11. Project.toml and Manifest.toml](#)



A successful approach to developing packages

Use *two* environments:

- ▶ The *package environment*
 - ▶ Dependencies: only those required by the package's code
- ▶ A “*playground*” environment – you work from here
 - ▶ Specify the package environment's path in the [sources] section of the `Project.toml` file
 - ▶ develop your package from here
 - ▶ Feel free to add any dependencies that may be useful for your development work

References:

- ▶ Modern Julia Workflows > Writing > [Development workflow](#)
- ▶ Pkg Documentation > 11. `Project.toml` and `Manifest.toml` > [The \[sources\] section](#)



Package extensions

A package extension is a module that is automatically loaded when *some other set of packages* (its “triggers”) are loaded into the Julia session.

Use cases:

- ▶ Replace heavy dependencies that not all users or tasks commonly require
- ▶ Provide additional functionality only when certain other packages are used together, while avoiding any dependency on them

References:

- ▶ Julia Documentation > Manual > Code Loading > [Package Extensions](#)
- ▶ Pkg Documentation > 5. Creating Packages > [Conditional loading of code in packages \(Extensions\)](#)

~~Writing fast code in Julia~~ Avoiding slow code in Julia

Jeroen Tant

KU Leuven ELECTA and Etch – EnergyVille

Writing Clean and Fast Code in Julia
IEEE PES Student Branch Leuven, May 6th, 2026

Steps to avoid slow code in Julia (content):

1. Use functions: Julia's compilation stages
2. Help type inference
3. Avoid unnecessary memory allocations
4. Further performance optimization starts now (if you still need it)
5. Use Benchmarking and Profiling tools (to know where to optimize)

This talk: understanding why code could run slow, by shaping a mental model of how Julia works.

Use functions: Julia's compilation stages

Use functions

Official performance tips:

<https://docs.julialang.org/en/v1/manual/performance-tips/>

“Performance critical code should be inside a function”.

Otherwise Julia's compiler cannot do anything...

Use Functions

Julia uses multiple dispatch:

- Functions in Julia can have multiple definitions; **called methods**
- At function calls, methods are **dispatched** based on types of provided arguments
- At first calls, **specialized method instances** are compiled for the provided arguments.

Hence:

- One function can have multiple method definitions
- One method can have multiple compiled method instances

```
julia> f(x,y) = x*y
f (generic function with 1 method)

julia> f(x::Int,y::Int) = x+y
f (generic function with 2 methods)

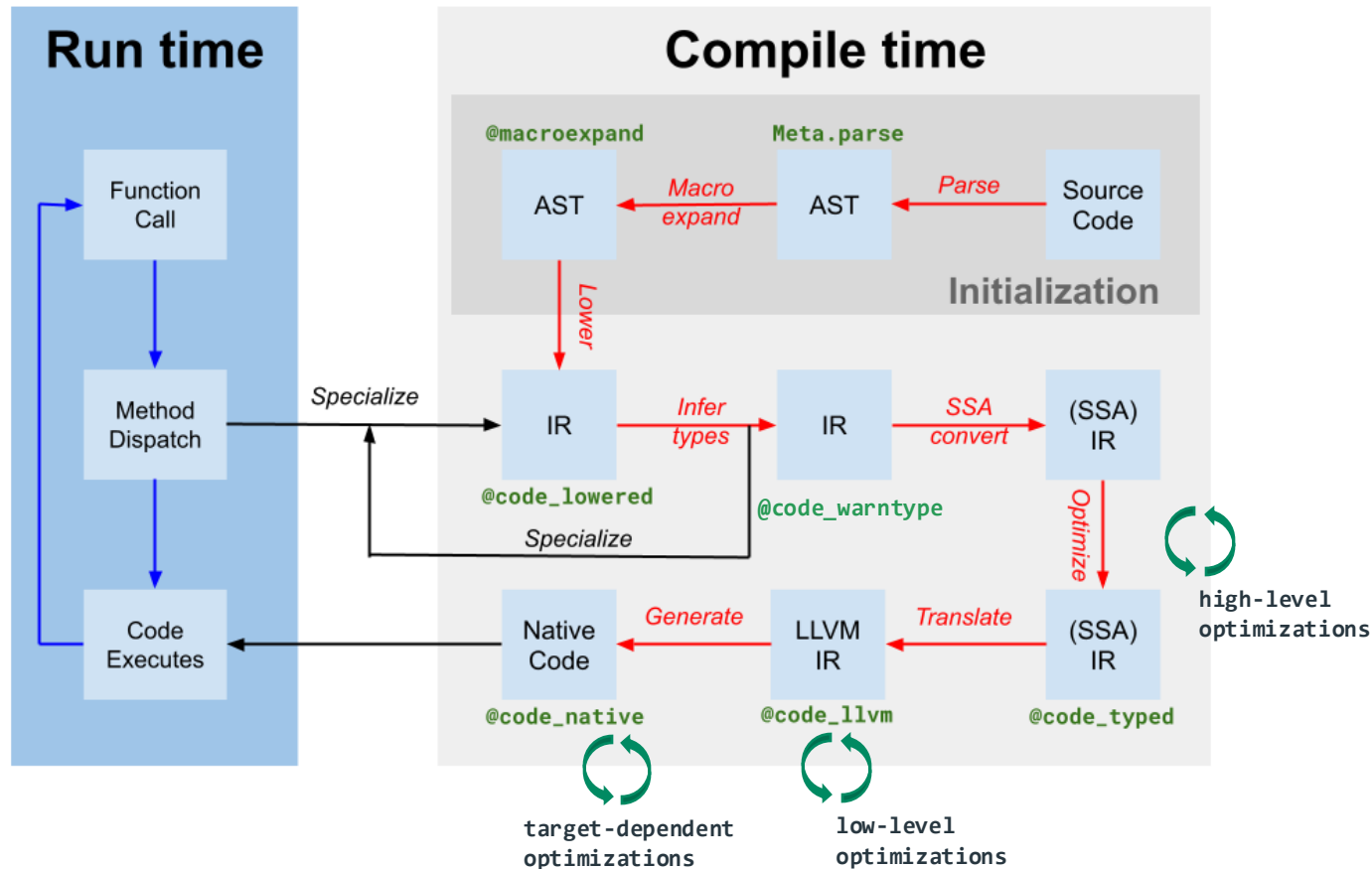
julia> f(1.0,2.0)
2.0

julia> f(1,2)
3

julia> f(1im,1im)
-1 + 0im
```

Julia's compilation stages

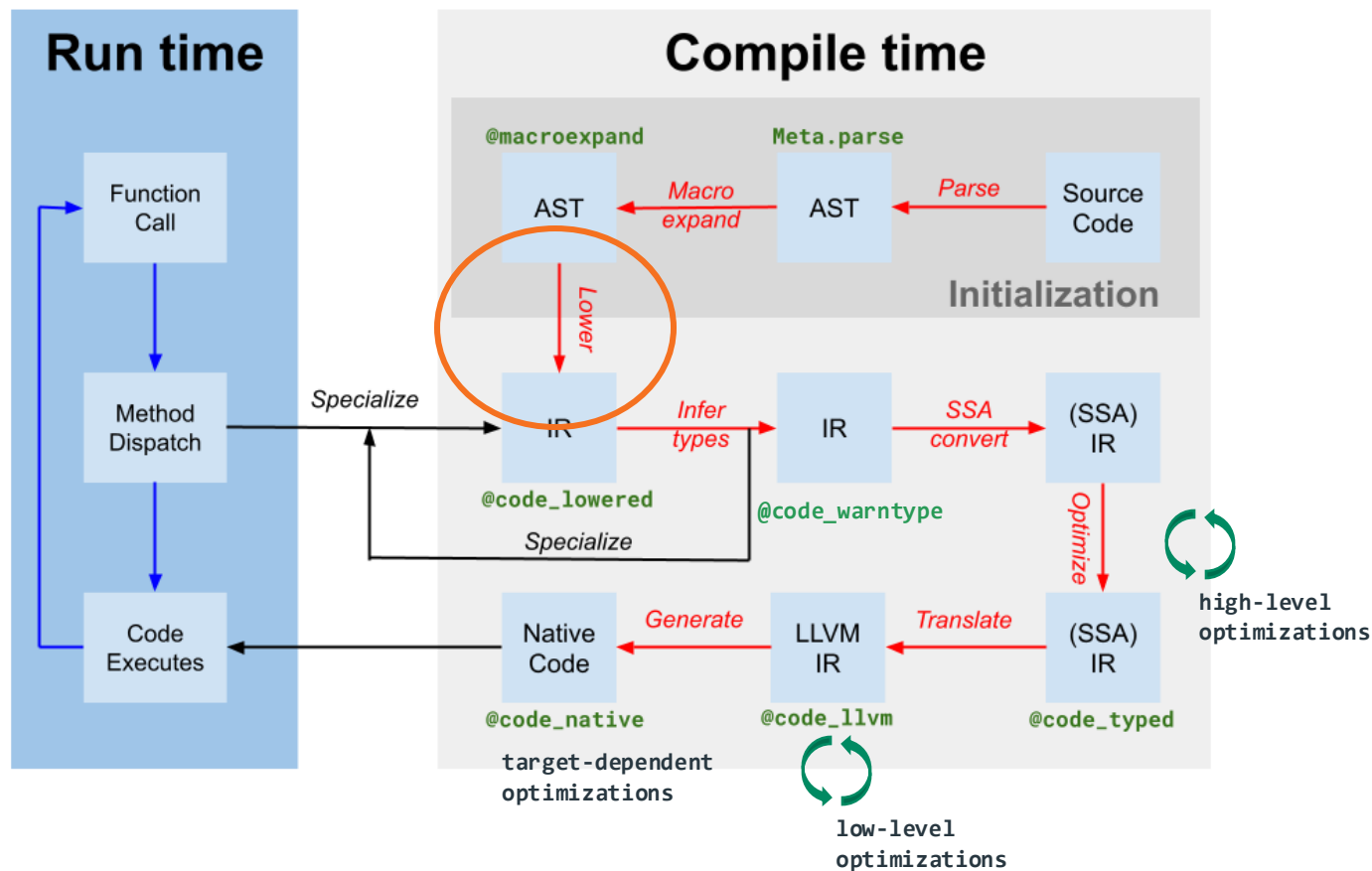
Source: <https://docs.julialang.org/en/v1/devdocs/eval/>



- Initialization time: evaluating code
- Compile time: when running a method instance for the first time
- Run time: actual execution of the compiled methodinstance

Julia's compilation stages

Source: <https://docs.julialang.org/en/v1/devdocs/eval/>



Initialization time: code lowering

- syntax transformation to a flat list of primitives that is easier to analyze and optimize
- control flow is transformed to explicit goto branches

```

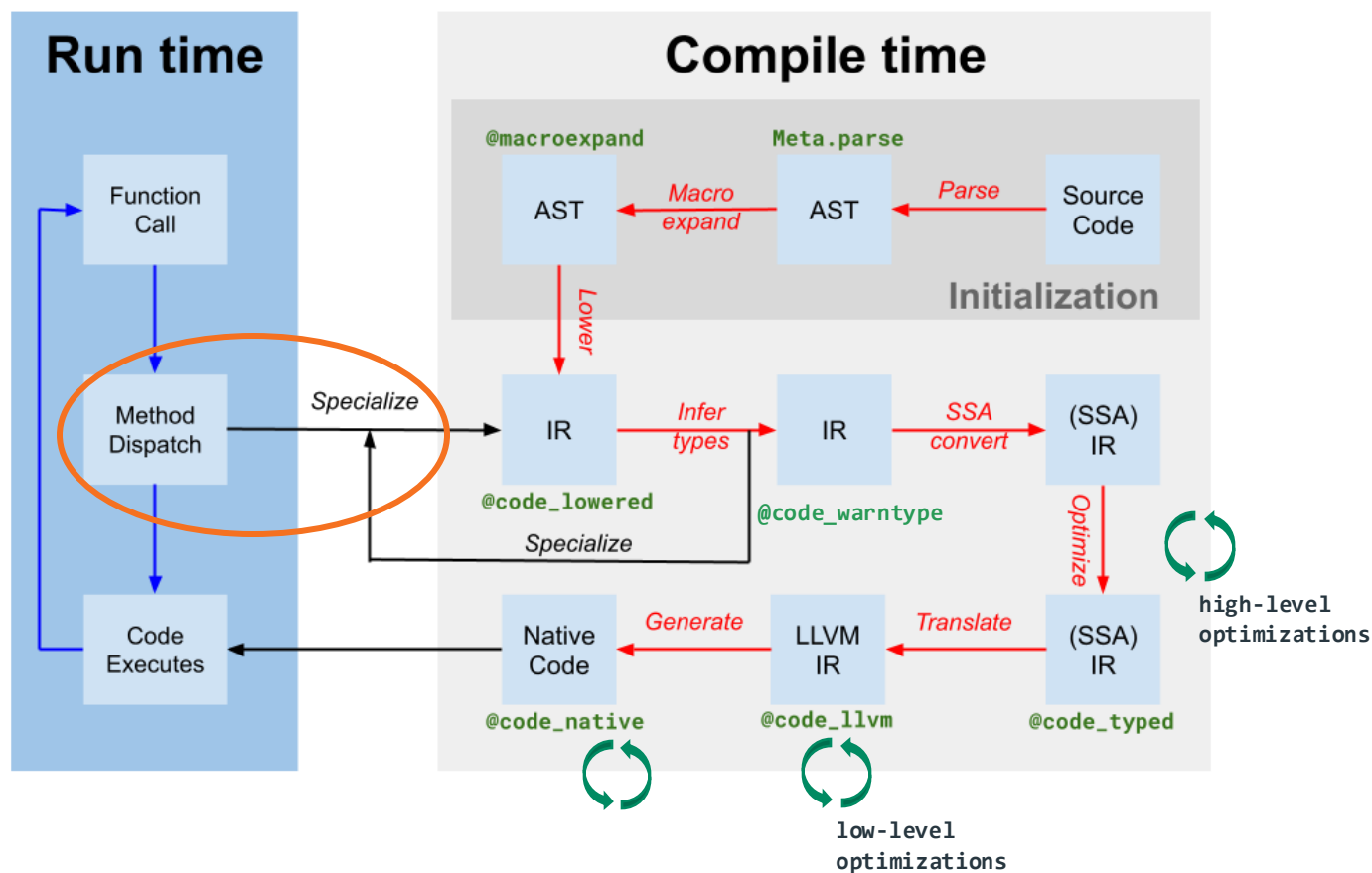
julia> function g(x,n)
    for i in 1:n
        x = x + 1
    end
    return x
end
g (generic function with 1 method)

julia> @code_lowered g(3,8)
CodeInfo(
1 -      x@_6 = x@_2
   %2  = Main.:(: )
   %3  = dynamic (%2)(1, n)
   @_4 = Base.iterate(%3)
   %5  = @_4
   %6  = builtin %5 == nothing
   %7  = dynamic Base.not_int(%6)
        goto #4 if not %7
2 ... %9  = @_4
      i = Core.getfield(%9, 1)
   %11 = builtin Core.getfield(%9, 2)
   %12 = Main.:+
   %13 = x@_6
        x@_6 = (%12)(%13, 1)
        @_4 = Base.iterate(%3, %11)
...

```

Julia's compilation stages

Source: <https://docs.julialang.org/en/v1/devdocs/eval/>

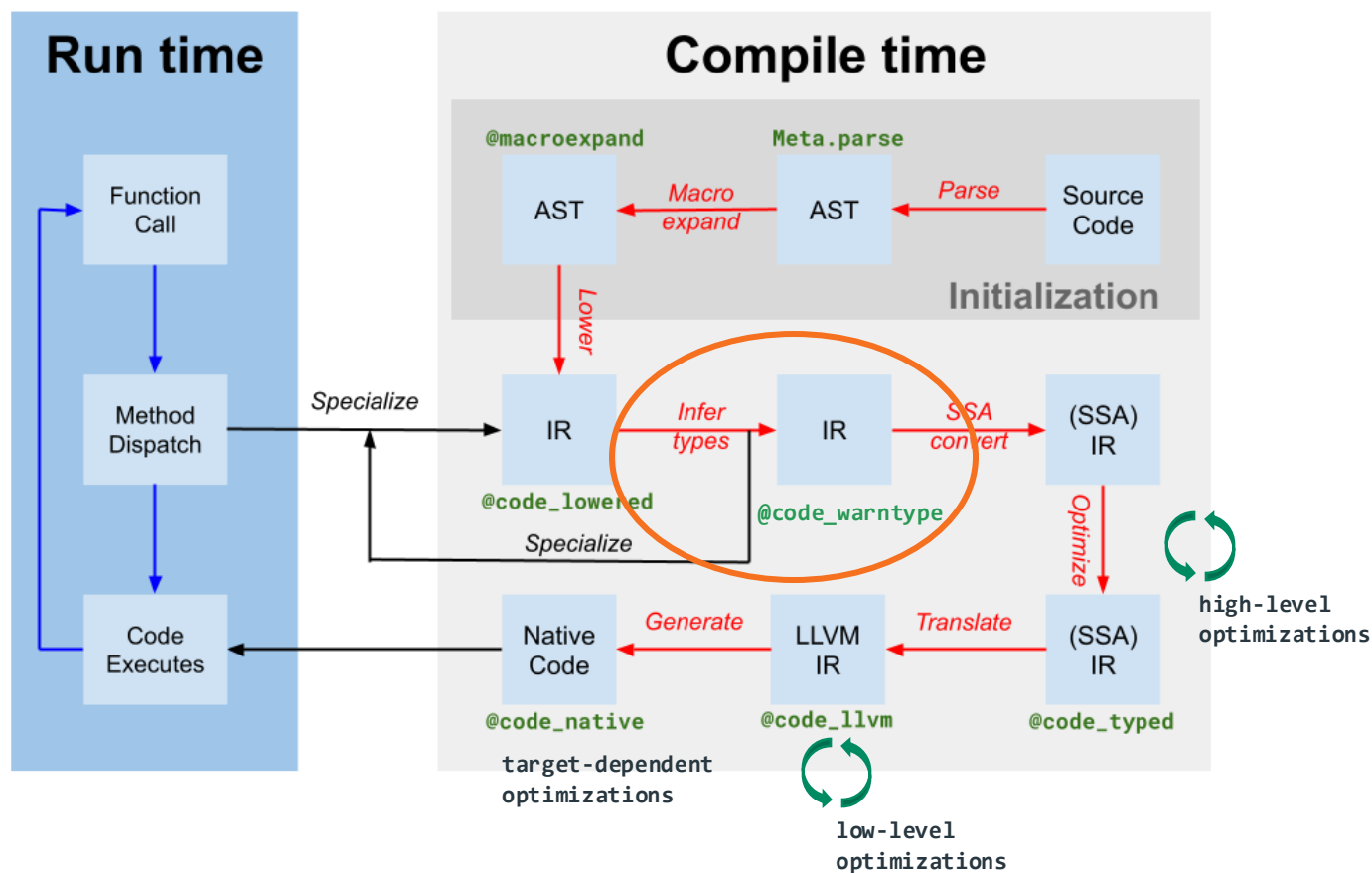


Run time; after method dispatch

- Methods can have multiple compiled method instances, specialized for the input arguments
- If this method instance was not encountered yet, it is compiled Just In Time (JIT).

Julia's compilation stages

Source: <https://docs.julialang.org/en/v1/devdocs/eval/>



Compile time: Type Inference

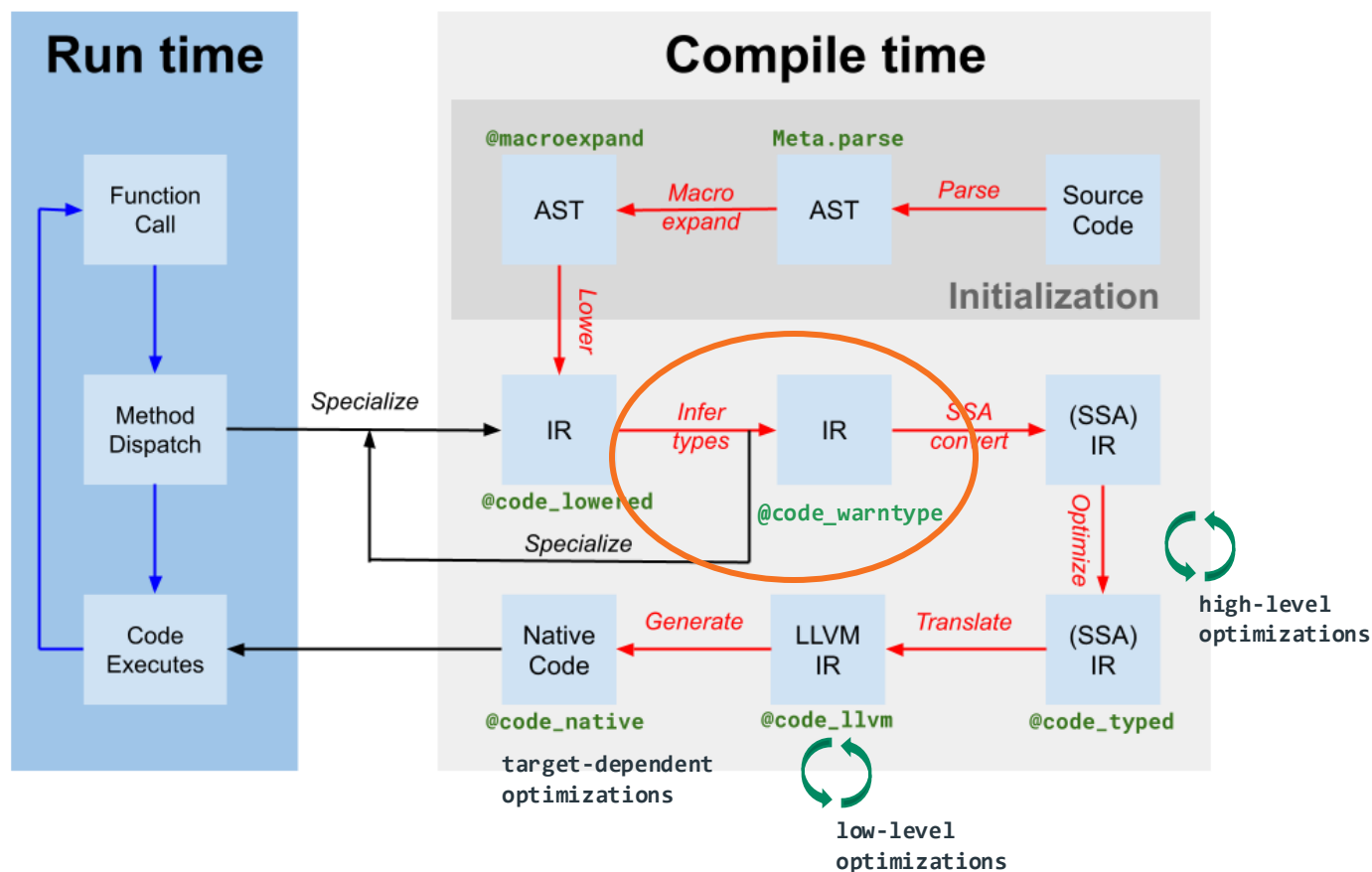
- Julia is a dynamic language; you do not have to specify types (= variables can have any type)
- But if Julia can infer types of variables, this enables static optimizations that bring the performance of static languages (C, C++, Fortran, ...)

```
julia> h(x) = 3*sin(x)
h (generic function with 2 methods)

julia> @code_warntype h(2)
MethodInstance for h(::Int64)
  from h(x) @ Main REPL[22]:1
Arguments
  #self#::Core.Const(Main.h)
  x::Int64
Body::Float64
1 - %1 = Main.*::Core.Const(*)
  %2 = Main.sin::Core.Const(sin)
  %3 = (%2)(x)::Float64
  %4 = (%1)(3, %3)::Float64
  return %4
```

Julia's compilation stages

Source: <https://docs.julialang.org/en/v1/devdocs/eval/>



Type Inference

- Julia is a dynamic language; you do not have to specify types (= variables can have any type)
- But if Julia can infer types of variables, this enables static optimizations that bring the performance of static languages (C, C++, Fortran, ...)

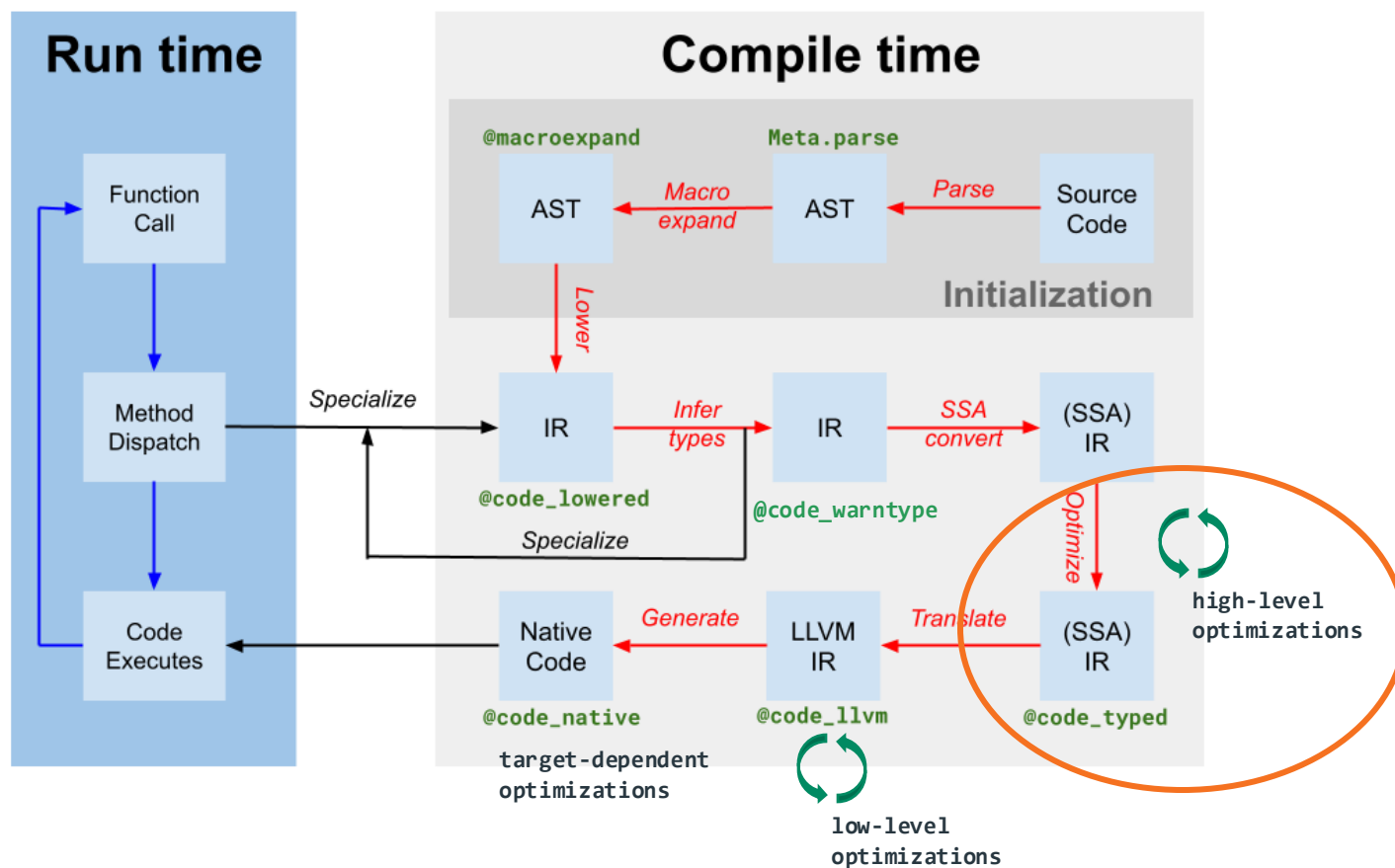
```
julia> myglobal = 2.0
2.0

julia> h() = 3*sin(myglobal)
h (generic function with 2 methods)

julia> @code_warntype h()
MethodInstance for h()
 from h() @ Main REPL[28]:1
Arguments
 #self#::Core.Const(Main.h)
Body::Any
1 - %1 = Main.*::Core.Const(*)
  %2 = Main.sin::Core.Const(sin)
  %3 = Main.myglobal::Any
  %4 = (%2)(%3)::Any
  %5 = (%1)(3, %4)::Any
  return %5
```

Julia's compilation stages

Source: <https://docs.julialang.org/en/v1/devdocs/eval/>



High-level optimizations, for instance:

- Function inlining
- Constant folding
-

```
julia> f(x,y) = x*y+1.0
f (generic function with 1 method)

julia> h(x) = 3.0*sin(x)
h (generic function with 1 method)

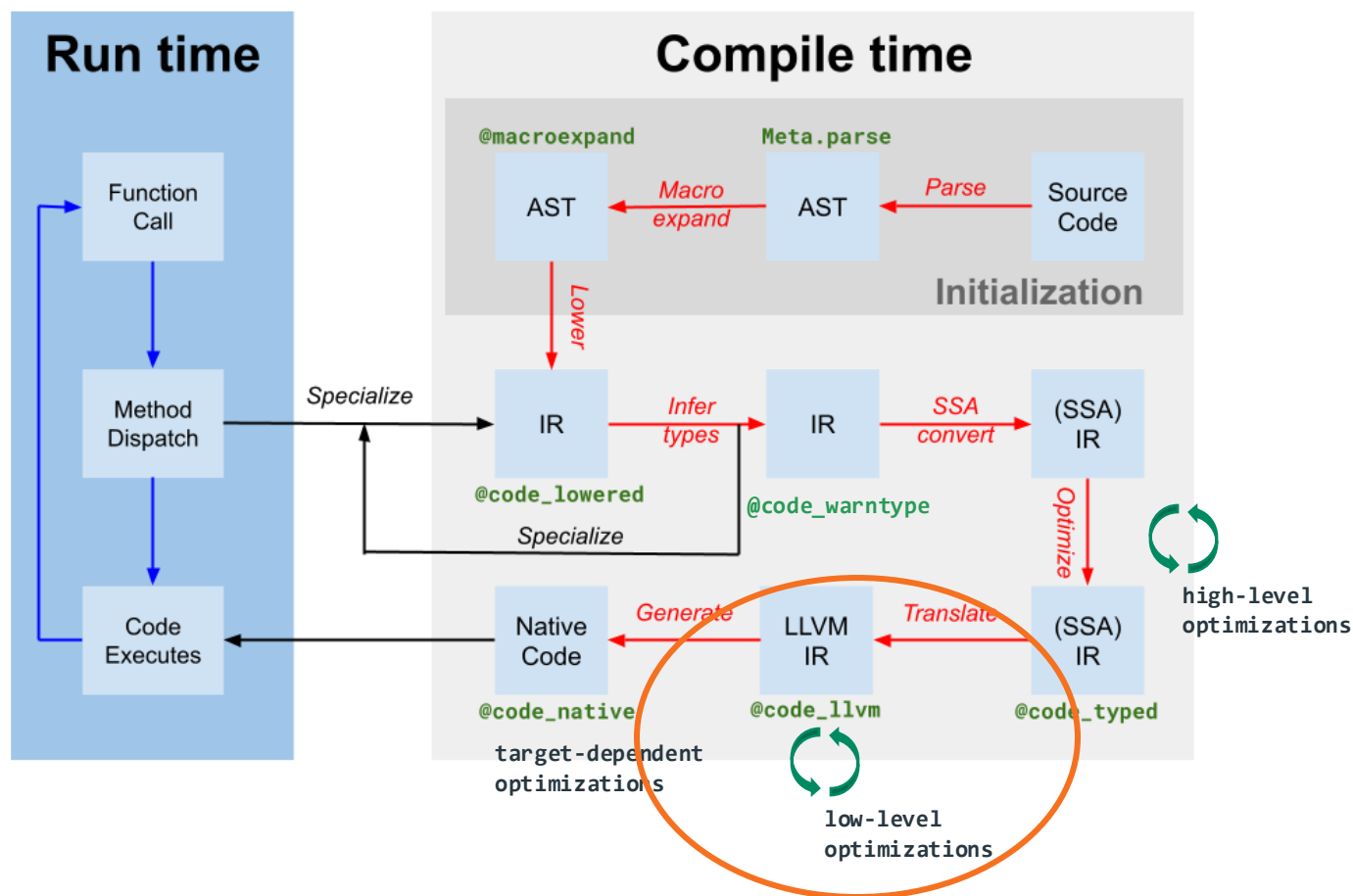
julia> g(x) = f(2.0,3.0)+h(x)
g (generic function with 1 method)

julia> @code_typed g(1.0)
CodeInfo(
  1 - %1 = invoke Main.sin(x::Float64)::Float64
    | %2 = intrinsic Base.mul_float(3.0, %1)::Float64
    | %3 = intrinsic Base.add_float(7.0, %2)::Float64
    | return %3
) => Float64
```

- `f` is constant-folded
- `h` is inlined
- `h` and `f` are not compiled and not called at runtime!

Julia's compilation stages

Source: <https://docs.julialang.org/en/v1/devdocs/eval/>



LLVM Compiler optimization passes, for instance:

- Dead code elimination (DCE)
- Scalar replacement of aggregates (SROA)
- Common subexpression elimination (CSE)
- Loop optimization: e.g., unrolling / vectorization / ...
- ... many others

```
julia> struct MyStruct
    x::Int
    y::Float64
end

julia> f(s::MyStruct) = s.x*sin(s.y)
f (generic function with 2 methods)

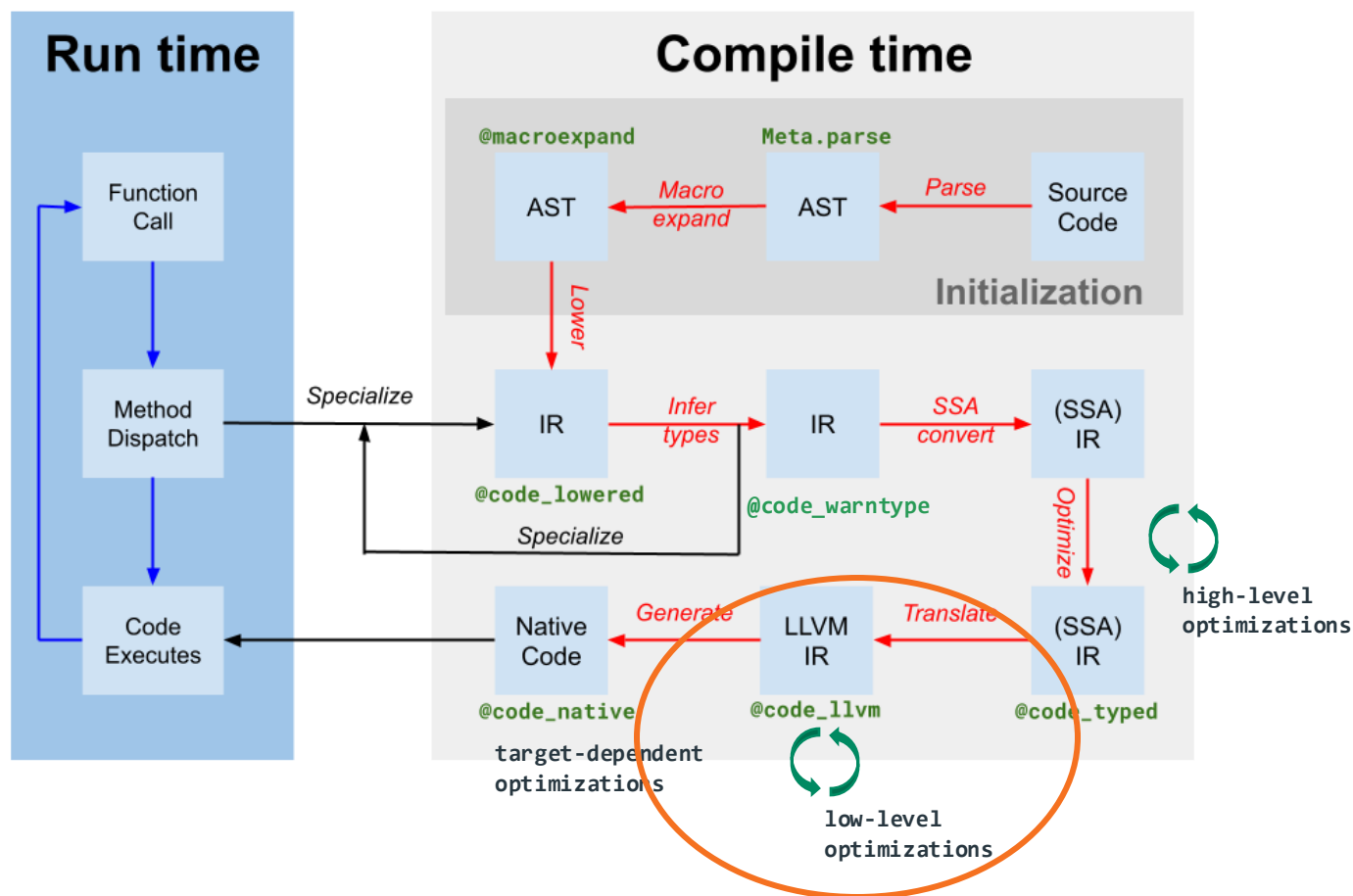
julia> g(z) = f(MyStruct(3,z))
g (generic function with 1 method)

julia> @code_llvm g(2.0)
; Function Signature: g(Float64)
; @ REPL[13]:1 within `g`
define double @julia_g_41987(double %"z::Float64") #0
{
top:
; @ REPL[12]:1 within `f`
%0 = call double @j_sin_41990(double %"z::Float64")
; @ promotion.jl:434 within `*` @ float.jl:497
%1 = fmul double %0, 3.000000e+00
ret double %1
; LL
}
```

MyStruct is never created! (SROA)

Julia's compilation stages

Source: <https://docs.julialang.org/en/v1/devdocs/eval/>



LLVM Compiler optimization passes, for instance:

- Dead code elimination (DCE)
- Scalar replacement of aggregates (SROA)
- Common subexpression elimination (CSE)
- Loop optimization: e.g., unrolling / vectorization / ...
- ... many others

```

julia> using StaticArrays

julia> v = randn(SVector{4,Float64},10);

julia> function addv!(v,x)
    for i in eachindex(v)
        v[i] = v[i].+x
    end
end
addv! (generic function with 1 method)

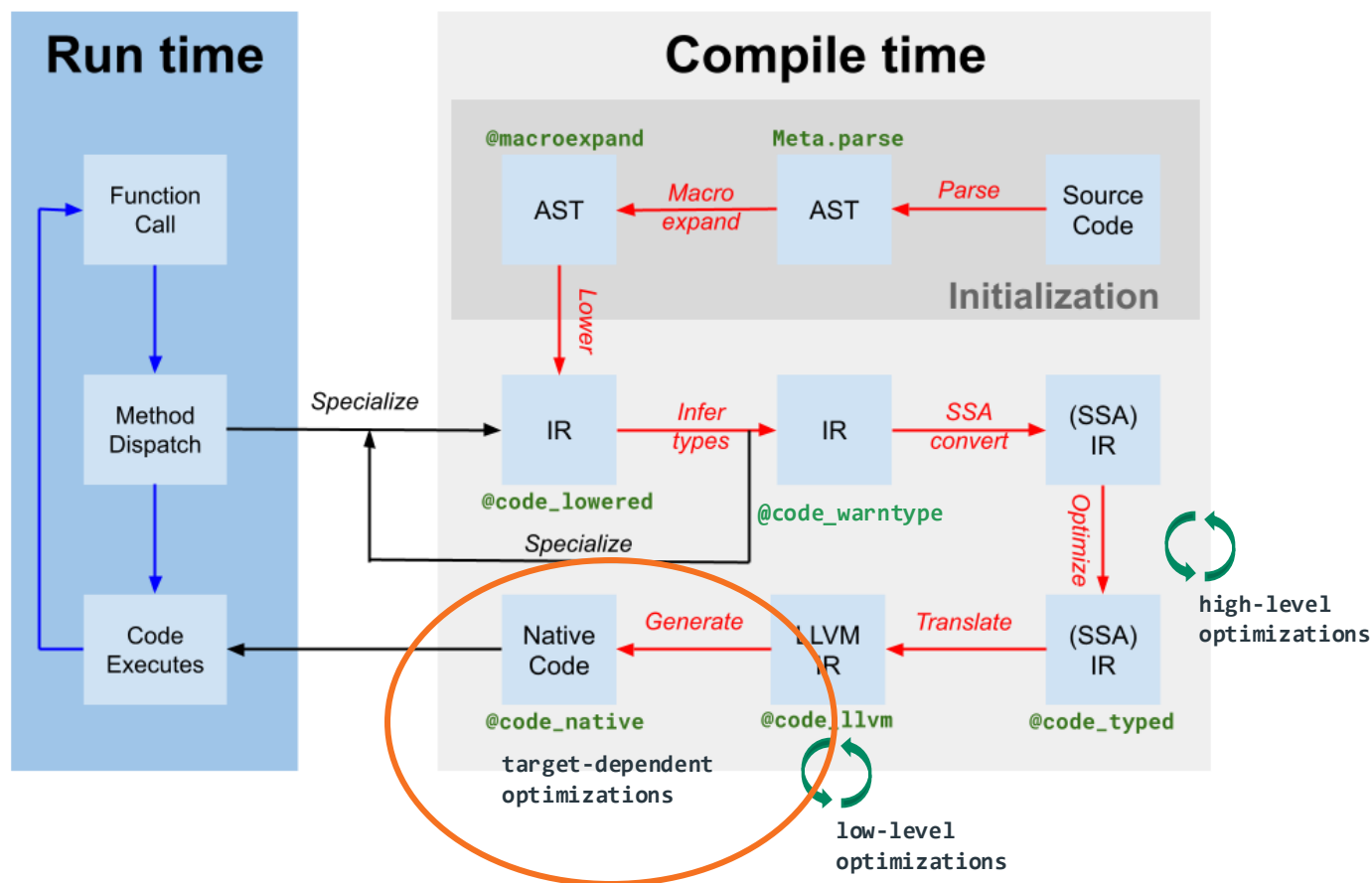
julia> @code_llvm addv!(v,(1,2,3,4))
...

        %6 = fadd <4 x double> %5, %2
; LLLLLL
; @ array.jl:986 within `setindex!`
; @ array.jl:991 within `_setindex!`
    store <4 x double> %6, ptr %memoryref_data7, align
8
...
  
```

SIMD Vectorization

Julia's compilation stages

Source: <https://docs.julialang.org/en/v1/devdocs/eval/>



Code generation:

- Machine code for target CPU
- Target-dependent optimizations (e.g. register assignment or CPU-specific instructions)

```
julia> @code_native addv!(v,(1,2,3,4))
...
    vcvtsi2sd    xmm1, xmm1, qword ptr [rsi + 16]
    vunpcklpd    xmm0, xmm1, xmm0          # xmm0 =
xmm1[0],xmm0[0]
    vcvtsi2sd    xmm1, xmm2, qword ptr [rsi + 8]
    vcvtsi2sd    xmm2, xmm2, qword ptr [rsi]
    vunpcklpd    xmm1, xmm2, xmm1          # xmm1 =
xmm2[0],xmm1[0]
    vinsertf128   ymm0, ymm1, xmm0, 1
    .p2align     4, 0x90
.LBB0_2:                                           # %L13
                                           # =>This Inner Loop
Header: Depth=1
; | @ REPL[39]:3 within `addv!`
; | r @ essentials.jl:919 within `getindex`
    cmp         rdx, qword ptr [rdi + 16]
    jae         .LBB0_5
    vaddpd       ymm1, ymm0, ymmword ptr [rcx]
...
```

Help Type Inference

Why type inference?

Functions calling other functions:

- Before applying high-level optimizations such as inlining and constant folding, which in turn enable low-level optimizations such as DCE, SROA, or vectorization, Julia needs to know the method instance of called functions
== **know function argument types at compile time.**
- If not: dynamic dispatch of methods at runtime.
- **Dynamic dispatch is not necessarily too slow; but the loss of optimization opportunity often is!**

```
julia> struct MyVal
    x
end

julia> f(s) = 3+sin(abs(s.x))
f (generic function with 1 method)
```

```
julia> @code_typed f(MyVal(1.0))
```

```
CodeInfo(
1 - %1 = builtin Base.getfield(s, :x)::Any
    %2 = dynamic Main.abs(%1)::Any
    %3 = dynamic Main.sin(%2)::Any
    %4 = dynamic (3 + %3)::Any
    return %4
) => Any
```

Dynamic dispatch
Dynamic dispatch
Dynamic dispatch

```
julia> struct MyVal2
    x::Float64
end

julia> f(s) = 3+sin(abs(s.x))
f (generic function with 1 method)
```

```
julia> @code_typed f(MyVal2(1.0))
```

```
CodeInfo(
1 - %1 = builtin Base.getfield(s, :x)::Float64
    %2 = intrinsic Base.abs_float(%1)::Float64
    %3 = invoke Main.sin(%2::Float64)::Float64
    %4 = intrinsic Base.add_float(3.0, %3)::Float64
    return %4
) => Float64
```

Main.abs inlined
static call (not inlined)
Base.+ inlined

Help type inference

#1 Avoid global variables unless defined with **const**

```
julia> myglobal = 2.0
2.0

julia> h() = 3*sin(myglobal)
h (generic function with 2 methods)

julia> @code_warntype h()
MethodInstance for h()
  from h() @ Main REPL[28]:1
Arguments
  #self#::Core.Const(Main.h)
Body::Any
1 - %1 = Main.*::Core.Const(*)
  | %2 = Main.sin::Core.Const(sin)
  | %3 = Main.myglobal::Any
  | %4 = (%2)(%3)::Any
  | %5 = (%1)(3, %4)::Any
  |   return %5
```

make types predictable

```
julia> const myglobal = 2.0
2.0

julia> h() = 3*sin(myglobal)
h (generic function with 1 method)

julia> @code_warntype h()
MethodInstance for h()
  from h() @ Main REPL[2]:1
Arguments
  #self#::Core.Const(Main.h)
Body::Float64
1 - %1 = Main.*::Core.Const(*)
  | %2 = Main.sin::Core.Const(sin)
  | %3 = Main.myglobal::Core.Const(2.0)
  | %4 = (%2)(%3)::Core.Const(0.9092974268256817)
  | %5 = (%1)(3, %4)::Core.Const(2.727892280477045)
  |   return
%5

julia> @code_typed h()
CodeInfo(
1 -      return 2.727892280477045 → Constant folded!
) => Float64
```

Help type inference

make types predictable

#2 Avoid containers with abstract element types unless you really need them

```
julia> using BenchmarkTools

julia> v = Any[1:100000...];

julia> typeof(v)
Vector{Any} (alias for Array{Any, 1})

julia> @btime sum($v)
1.206 ms (99969 allocations: 1.53 MiB)
5000050000
```

```
julia> using BenchmarkTools

julia> v = Int64[1:100000...];

julia> typeof(v)
Vector{Int64} (alias for Array{Int64, 1})

julia> @btime sum($v)
6.411 μs (0 allocations: 0 bytes)
5000050000
```

Help type inference

make types predictable

#3 Avoid structs with abstract fields (use concrete types or **type parameters** instead!)

```
julia> struct MyVal
    x
end

julia> f(s) = 3+sin(abs(s.x))
f (generic function with 1 method)

julia> @code_typed f(MyVal(1.0))
CodeInfo(
  1 - %1 = builtin Base.getfield(s, :x)::Any
    %2 = dynamic Main.abs(%1)::Any
    %3 = dynamic Main.sin(%2)::Any
    %4 = dynamic (3 + %3)::Any
    return %4
) => Any
```

```
julia> struct MyVal2
    x::Float64
end

julia> f(s) = 3+sin(abs(s.x))
f (generic function with 1 method)

julia> @code_typed f(MyVal2(1.0))
CodeInfo(
  1 - %1 = builtin Base.getfield(s, :x)::Float64
    %2 = intrinsic Base.abs_float(%1)::Float64
    %3 = invoke Main.sin(%2::Float64)::Float64
    %4 = intrinsic Base.add_float(3.0, %3)::Float64
    return %4
) => Float64
```

```
julia> struct MyVal3{T}
    x::T
end

julia> f(s) = 3+sin(abs(s.x))
f (generic function with 1 method)

julia> @code_typed f(MyVal3{Float64}(1.0))
CodeInfo(
  1 - %1 = builtin Base.getfield(s, :x)::Float64
    %2 = intrinsic Base.abs_float(%1)::Float64
    %3 = invoke Main.sin(%2::Float64)::Float64
    %4 = intrinsic Base.add_float(3.0, %3)::Float64
    return %4
) => Float64
```

Help type inference

make types predictable

#4 Write type-stable functions: ensure output type is predictable from input types and not dependent on input **values**

#5 Ensure variables within a function have only one type

```
julia> function notstable(x)
    if x > 0
        z = sin(x)
    else
        z = 0
    end
    return 3*z
end
notstable (generic function with 1 method)
```

Type of z depends on value of x

```
julia> @code_warntype notstable(1.0)
MethodInstance for notstable(::Float64)
  from notstable(x) @ Main REPL[1]:1
Arguments
  #self#::Core.Const(Main.notstable)
  x::Float64
Locals
  z::Union{Float64, Int64}
Body::Union{Float64, Int64}
1 -      Core.NewvarNode(:z)
   | %2 = Main.:>::Core.Const(>)
   | %3 = (%2)(x, 0)::Bool
   |      goto #3 if not %3
2 - %5 = Main.sin::Core.Const(sin)
   |      (z = (%5)(x))
   |      goto #4
3 -      (z = 0)
4 - %9 = Main.:*::Core.Const(*)
   | %10 = z::Union{Float64, Int64}
   | %11 = (%9)(3, %10)::Union{Float64, Int64}
   |      return %11
```

```
julia> @code_typed notstable(1.0)
CodeInfo(
  1 - %1 = intrinsic Base.lt_float(0.0,
  x)::Bool
  | %2 = intrinsic Base.or_int(%1,
  false)::Bool
  |      goto #3 if not %2
  2 - %4 = invoke
  Main.sin(x::Float64)::Float64
  |      goto #4
  3 -      nothing::Nothing
  4 - %7 = φ (#2 => true, #3 => false)::Bool
  | %8 = φ (#2 => %4, #3 =>
  0)::Union{Float64, Int64}
  |      goto #6 if not %7
  5 - %10 = π (%8, Float64)
  | %11 = intrinsic Base.mul_float(3.0,
  %10)::Float64
  |      goto #7
  6 -      nothing::Nothing
  | %14 = π (%8, Int64)
  | %15 = intrinsic Base.mul_int(3,
  %14)::Int64
  |      goto #7
  7 - %17 = φ (#5 => %11, #6 =>
  %15)::Union{Float64, Int64}
  |      return %17
) => Union{Float64, Int64}
```

Help type inference

make types predictable

#6 If you do not know variable types beforehand: use **function barriers**, that is: dispatch (dynamically) as soon as possible in high-level code so that lower-level code (hot loops) is type-aware. (<https://docs.julialang.org/en/v1/manual/performance-tips/#kernel-functions>)

```
julia> using BenchmarkTools

julia> function f_work(vs)
    r = 0.0
    for v in vs
        for i in eachindex(v)
            r += iseven(i) ? v[i] : -v[i]
        end
    end
    return r
end
f_work (generic function with 1 method)

julia> vs = Vector[rand(Int,2000), rand(Float64,2000),
rand(Int8,200)];

julia> @btime f_work($vs)
131.783 μs (13178 allocations: 205.91 KiB)
```

```
julia> using BenchmarkTools

julia> function f_barrier(v::Vector{T}) where {T}
    r = zero(T)
    for i in eachindex(v)
        r += iseven(i) ? v[i] : -v[i]
    end
    return r
end
f_barrier (generic function with 1 method)

julia> function f_work2(vs)
    r = 0.0
    for v in vs
        r += f_barrier(v)
    end
    return r
end
f_work2 (generic function with 1 method)

julia> vs = Vector[rand(Int,2000), rand(Float64,2000),
rand(Int8,200)];

julia> @btime f_work2($vs)
1.397 μs (5 allocations: 80 bytes)
```

Avoid unnecessary memory allocations

Memory Allocations

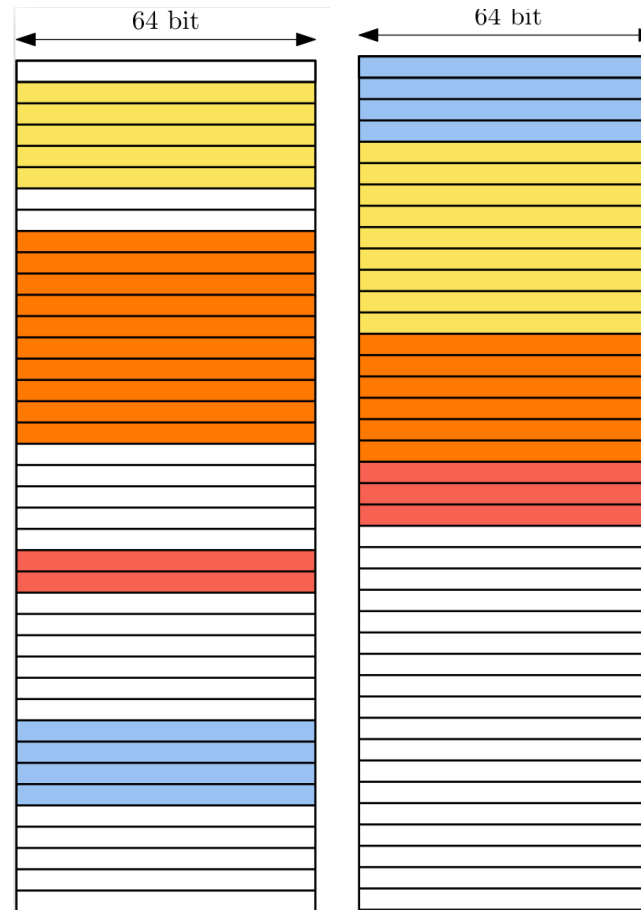
The OS provides each Julia process with a **private virtual address space**:

- Each process uses portions of this virtual address space for **the heap**, where dynamic memory allocation (malloc) occurs.
- Each thread in a process uses portions of this virtual address space for **the stack**, which is used for function call frames and stack allocations.

Heap vs Stack

Heap:

- **Dynamic allocation** of objects,
- Object size may not be known at compile time.
- Requires finding or reusing a suitable memory region (**allocation cost**).
- Objects must be tracked and reclaimed when no longer used: manual (C/C++) or **garbage collection** (Julia).



Stack:

- **LIFO** (last-in, first-out)
- Fast allocation of **objects local to a function call** by adjusting the stack pointer
- Object sizes are known at compile time
- Objects always fit as long as the stack is not full (stack overflow).
- Objects are freed automatically when the function returns: **no garbage collection required**

Where does Julia store object values?

The user does not control this, Julia can store object values on:

- the heap (fixed address, needs allocation and GC tracking),
- the stack (temporary address),

or can choose not to store object values in memory at all:

- because they exist only temporarily in CPU registers (no address),
- or because the object is split by the compiler into separate values for each field (SROA - Scalar Replacement of Aggregates).

When does Julia store objects on the heap (allocations)?

Mutable structs (including arrays):

- Values can be mutated -> there should be only one version, and this version needs a unique identifier (i.e., a stable memory address).
- Typically on the heap (stable memory address), requiring GC. -> **Allocation!**
- But the compiler can also choose to allocate it on the stack, if it can prove that the object cannot escape a function call (this is "escape analysis"; improved a lot in recent Julia versions).

Immutable structs and Primitive types (e.g., Int, Float64, ...):

- Value cannot change -> Julia is free to store multiple versions at multiple locations.
- Struct objects: typically, stored on the stack if not too big and often split into its fields (SROA).
- Primitive types: typically, stored on the stack / in registers

When does Julia store objects on the heap (allocations)?

Array assignments:

- Direct inline storage of immutable structs or primitives: in the already allocated contiguous memory region of the array (only for concrete pointer-free "isbitstype" element types; or small unions of such element types)
- Indirect storage of a pointer to a mutable struct: only the pointer is stored, and the mutable struct was already allocated before.
- Indirect storage of a pointer to an immutable struct: the value is "boxed" and allocated on the heap. -> Allocation! (e.g., when storing values inside `Vector{Any}`, or when storing non-isbits structs)

Field assignments of a mutable struct:

- Direct inline storage of an immutable struct or primitives: in the already allocated contiguous memory of the mutable struct.
- Direct inline storage of pointers to mutable structs
- Indirect storage of an immutable struct: the value is "boxed" and allocated on the heap. Only a pointer to the object is stored in the array.
-> Allocation! (e.g., when storing values in a field with abstract type)

Avoiding (heap) memory allocations?

- You cannot avoid **all** memory allocations, they are needed and extremely useful.
- Julia uses pools to allocate small objects (≤ 2032 bytes) faster.
- The cost of **allocating** memory is actually quite small, especially when the memory it is not yet used! But it can be **relatively** large and other hidden costs may show up later: memory traffic and garbage-collection pressure.
- Therefore, avoidable **repeated allocations** can make code slow(er) than necessary, and those are the kind of allocations that should be avoided.

< 2032 bytes

```
julia> @btime Vector{Int}(undef,250);  
15.219 ns (2 allocations: 2.02 KiB)
```

> 2032 bytes

```
julia> @btime Vector{Int}(undef,260);  
51.932 ns (3 allocations: 2.13 KiB)
```

Avoiding (heap) memory allocations?

#1 avoid repeated allocation of mutable structs and arrays

$x = a * x$ allocates

```
julia> function do_iter(a,x,n)
    for i in 1:n
        x = a*x
    end
    return x
end
do_iter (generic function with 1 method)

julia> x = randn(1000);

julia> do_iter(3.0,x,1);

julia> @time do_iter(3.0,x,1_000_000);
0.725746 seconds (3.00 M allocations: 7.518
GiB, 31.39% gc time)
```

Fused $x. = a.*x$ does not allocate

```
julia> function do_iter(a,x,n)
    for i in 1:n
        x .= a.*x
    end
    return x
end
do_iter (generic function with 1 method)

julia> x = randn(1000);

julia> do_iter(3.0,x,1);

julia> @time do_iter(3.0,x,1_000_000);
0.059382 seconds
```

Note: in this example, allocation time per iteration is relatively large compared to work per iteration. There is also indirect GC time measured (31 %), which is required to cleanup objects.

Avoid memory allocations

Avoid repeated assignment into arrays with Any or abstract element types.

Because element size is unknown a priori, only memory addresses of elements are stored in the array. Each element that does not have a fixed memory location yet must be "boxed" and allocated on the heap.

```
julia> function fillvector(v)
    for i in eachindex(v)
        v[i] = rand{Int}
    end
end
fillvector (generic function with 1 method)

julia> v1 = Vector{Any}(undef,1000);

julia> @allocations fillvector(v1)
1000

julia> @btime fillvector(v1)
5.255 μs (1000 allocations: 15.62 KiB)
```

```
julia> function fillvector(v)
    for i in eachindex(v)
        v[i] = rand{Int}
    end
end
fillvector (generic function with 1 method)

julia> v2 = Vector{Int}(undef,1000);

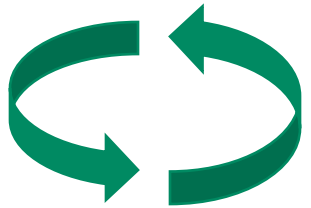
julia> @allocations fillvector(v2)
0

@btime fillvector(v2)
597.825 ns (0 allocations: 0 bytes)
```

**Further performance
optimization starts
now
(if you still need it)**

Performance optimization

Be pragmatic: optimize **total** development and user time,
(especially when developer = user)



- Design time (thinking)
- Implementation time (programming)
- Setup / configuration time (usability)
- Compilation time (JIT and package pre-compilation)
- Run time (topic of this talk)
- Garbage collection time
- Result interpretation time

Performance optimization

Run-time optimization (in order of importance):

1. **Measure**
(profiling & benchmarking to find real bottlenecks; avoid premature optimization)
2. **Algorithms & data-structure**
(big O time / memory complexity matters: e.g., $O(n^2)$ vs. $O(n)$ vs. $O(\log(n))$ vs. $O(1)$)
3. **Type-stability in bottleneck code**
(predictable types enable compiler optimizations)
4. **Avoid unnecessary allocations**
(especially repeated allocations in hot loops)
5. **Parallelization**
(multi-processing or shared-memory multi-threading)
6. **Cache-locality**
(predictable, reusable, sequential memory access patterns)
7. **Advanced code-optimizations**
(e.g., explicit SIMD tuning, cache aware optimizations,)

Performance optimization

Cache locality

- CPUs have L1, L2, L3 caches
- Accessing cache memory is **much faster** than accessing main memory
- Data is transferred in **cache lines** (64 bytes $\approx$ 8 x Float64)
- CPUs can **prefetch multiple cache lines** ahead (detecting memory streams)
- CPUs use **prediction** to mitigate memory latency
- **Predictable, sequential memory access** help the CPU to prefetch effectively (spatial locality)
- Reusing data while it is still in cache makes code faster (temporal locality)

Performance optimization

Cache locality example: Julia's matrices are stored column-wise

Column-wise loop

```
julia> using BenchmarkTools

julia> function sum_elements_v1(A)
    s = 0.0
    for j in axes(A,2)
        for i in axes(A,1)
            s += A[i,j]
        end
    end
    return s
end

sum_elements_v1 (generic function with 1 method)

julia> A = randn(10000,10000);

julia> @btime sum_elements_v1($A);
63.267 ms (0 allocations: 0 bytes)
```

Row-wise loop

```
julia> using BenchmarkTools

julia> function sum_elements_v2(A)
    s = 0.0
    for i in axes(A,1)
        for j in axes(A,2)
            s += A[i,j]
        end
    end
    return s
end

sum_elements_v2 (generic function with 1 method)

julia> A = randn(10000,10000);

julia> @btime sum_elements_v2($A);
324.222 ms (0 allocations: 0 bytes)
```

5x slower

Benchmarking and Profiling Tools

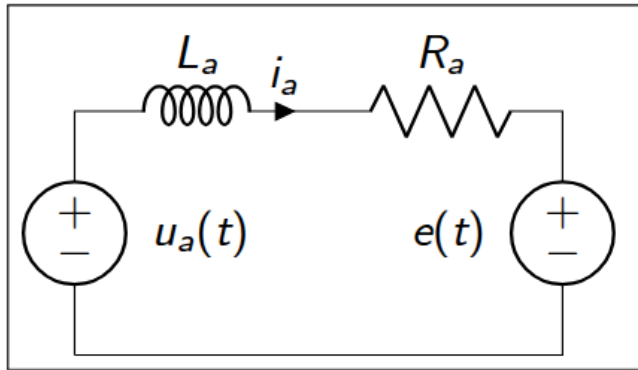
Benchmarking and Profiling Tools

- Julia
 - @time: execute expression once and measure time
 - @code_warntype: inspect type stability (inference results)
 - @code_typed: inspect static/dynamic dispatch, inlining
 - @code_llvm: inspect if vectorization is applied
- BenchmarkTools.jl
 - @benchmark: execute expression multiple times and give time statistics
 - @btime: execute expression multiple times and give minimal runtime
- In VS Code
 - @profview: profile code to discover critical sections and bottlenecks.
 - @profview_allocs: profile code to discover code sections that allocate.
- Outside VS Code: ProfileView.jl

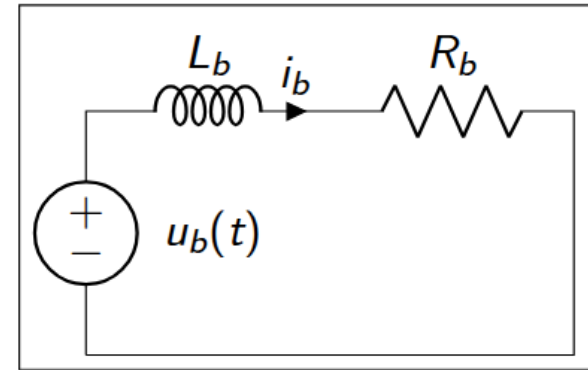
Demo

Demo model

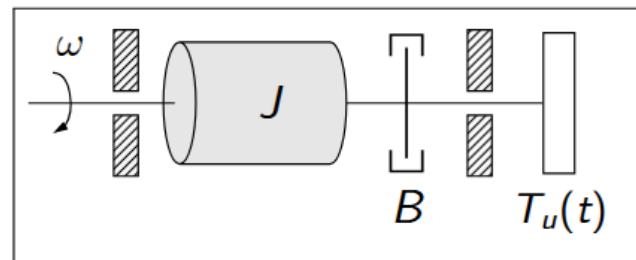
DC motor with independent excitation drives time-dependent load



(a) Motor anchor



(b) Motor excitation winding



(c) Mechanical load

Demo model

Equations of the system

J	Moment of inertia	R	Resistance
B	Damping coefficient	L	Inductance
T	Torque	u	Voltage
ω	Angular velocity	i	Current
k	Motor constant	e	Electromotive force

$$J \frac{d\omega}{dt} = -B\omega - T_u(t) + ki_a i_b \quad \text{Newton} \quad (1a)$$

$$L_a \frac{di_a}{dt} = u_a k i_b \omega - R_a i_a \quad \text{Kirchhoff (anchor)} \quad (1b)$$

$$L_b \frac{di_b}{dt} = u_b - R i_b \quad \text{Kirchhoff (excitation)} \quad (1c)$$

The pillars of clean code

Correctness

- ▶ Algorithm behaves as expected
- ▶ Garbage in, error/warning out

Reusability

- ▶ Reuse parts of the code elsewhere (code as a library)
- ▶ Reuse entire scripts for specific tasks (code as a binary)

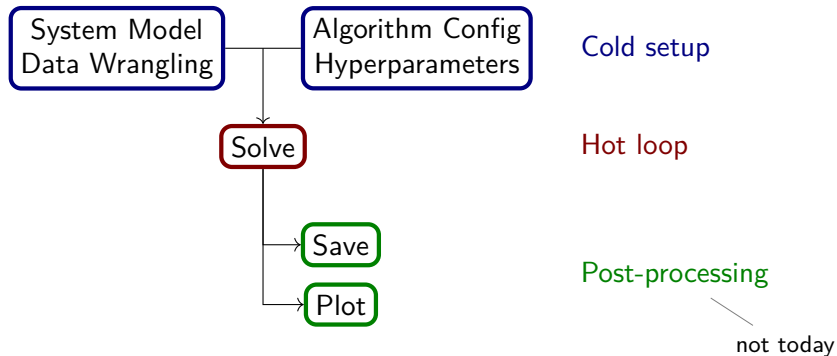
Readability

- ▶ Clear structure
- ▶ Documentation/comments/names...

Extensibility

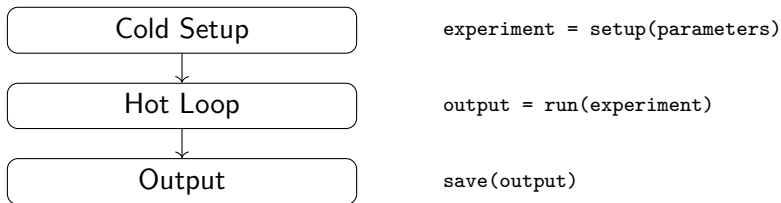
- ▶ Add new functionality easily without breaking existing code
- ▶ Do not overly restrict function usage

Scientific computing broadly follows the same structure





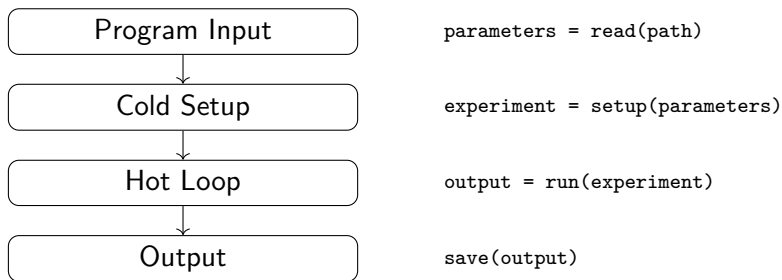
Config files make your code more reusable





user_input

Config files make your code more reusable



- ▶ code = protocol for executing experiment
- ▶ config file = what experiment to execute



user_input

Taking commandline arguments

```
$ julia --project run.jl --output result.csv --no-plot system.yaml
```

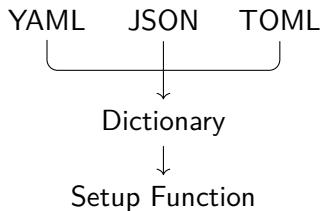


Taking commandline arguments

```
$ julia --project run.jl --output result.csv --no-plot system.yaml
```

- ▶ Where to get inputs
- ▶ Where to store outputs
- ▶ Enable/disable certain features
- ▶ [ArgParse.jl](#) for parsing arguments
- ▶ Improves reusability of scripts

Working with config files



```
function read_config(path)
  extensions = Dict(
    "json" => JSON.parsefile,
    "yaml" => YAML.load_file,
    "yml"  => YAML.load_file,
    "toml" => TOML.parsefile,
  )
  extensions[split(path, ".") |> last](path)
end
```

Limitations of script reuse in Julia

- ▶ Julia scripts and binaries are (prohibitively) slow to startup
- ▶ For large batch jobs: take a list of inputs/outputs as argument and loop

Benchmarking $1 + 1$

```
# with startup
```

```
$ time for i in $(seq 1 100); do julia -e '1+1'; done
      real    0m36.334s
```

```
# without startup, not yet compiled
```

```
julia> @time map(x-> x + 1, ones(100));
0.023485 seconds (47.78 k allocations: 2.295 MiB, 99.54% compilation time)
```

```
# without startup, already compiled
```

```
julia> f(x) = x + 1; f(1.0); @time map(f, ones(100));
0.000001 seconds (2 allocations: 928 bytes)
```

System modeling is our core business and it's hard

- ▶ Be robust against modeling mistakes (i.e. garbage in, error out)
- ▶ Different physical/digital components to model
- ▶ Different ways to model the same components
- ▶ Different ways to compose components into a larger system
- ▶ Different tasks to do (simulation, optimisation, state estimation)

System modeling is our core business and it's hard

- ▶ **Be robust against modeling mistakes (i.e. garbage in, error out)**
- ▶ Different physical/digital components to model
- ▶ Different ways to model the same components
- ▶ Different ways to compose components into a larger system
- ▶ Different tasks to do (simulation, optimisation, state estimation)

Keyed data storage in a nutshell

Aspect	Dict	NamedTuple	Struct
Mutability	Yes	No	Both
Value types	Homogeneous	Heterogeneous	Heterogeneous
Data validation	No	No	Yes
Multiple dispatch	No	Partly	Yes
Modeling use case	Almost never	Tabular data	Everything else

Keyed data storage in a nutshell

Aspect	Dict	NamedTuple	Struct
Mutability	Yes	No	Both
Value types	Homogeneous	Heterogeneous	Heterogeneous
Data validation	No	No	Yes
Multiple dispatch	No	Partly	Yes
Modeling use case	Almost never	Tabular data	Everything else

Dicts do not suck in general, they just suck for modeling

Generic data containers are easy to misuse and hard to use properly

```
linear(data, x) = data[:a] * x + data[:b]
```

```
quadratic(data, x) = data[:a] * x^2 + data[:b] * x + data[:c]
```

```
# likely we meant quadratic here, or we meant to use another config
```

```
julia> linear(Dict{:a => 1, :b => 1, :c => 1}, 1)
```

```
2
```

```
# impossible to specialise on
```

```
linear_is_constant(data) = data[:a] == 0
```

```
quadratic_is_constant(data) = data[:a] == 0 && data[:b] == 0
```

Custom structs allow data validation and dispatching

```
julia> struct Linear{T}
    a::T
    b::T
end
```

```
julia> Linear(;a, b) = Linear(a, b)
```

```
julia> (f::Linear)(x) = f.a * x + f.b
```

```
# the user error is caught!
```

```
julia> Linear(;Dict{:a => 1, :b => 1, :c => 1}...)
```

```
ERROR: MethodError: no method matching Linear(; a::Int64, b::Int64, c::Int64)
```

```
# multiple dispatch allows polymorphism
```

```
julia> is_constant(f::Linear) = f.a == 0
```

```
julia> is_constant(f::Quadratic) = f.a == 0 && f.b == 0
```

Data validation in constructors

- ▶ Constructors can cascade into each other
- ▶ Inner constructors are always respected, and are perfect for data validation

```
"""Grade normalised to a 20-point scale"""  
struct Grade  
    score::Int  
    function Grade(score) # this is an inner constructor  
        # every Grade instance is validated always, everywhere in your code  
        @assert score in 0:20  
        new(score)  
    end  
end  
# Additional (outer) constructors go here  
Grade(grade::Int, feedback) = Grade(grade) # disregard feedback
```

Take home messages

- ▶ Use structs
- ▶ Validate your input data in inner constructors

System modeling is our core business and it's hard

- ▶ Be robust against modeling mistakes (i.e. garbage in, error out)
- ▶ **Different physical/digital components to model**
- ▶ **Different ways to model the same components**
- ▶ Different ways to compose components into a larger system
- ▶ Different tasks to do (simulation, optimisation, state estimation)



Create smaller components to build bigger components

- Structs model physical components of the DC motor

```
struct Anchor{T}
  R::T
  L::T
  k::T
  function Anchor(R::T, L::T,
    k::T) where {T}
    @assert R > 0 && L > 0 && k > 0
    new{T}(R, L, k)
  end
end
```

```
struct ExcitationWinding{T}
  R::T
  L::T
  function ExcitationWinding(
    R::T, L::T) where {T}
    @assert R > 0 && L > 0
    new{T}(R, L)
  end
end
```



Compose building blocks for more flexibility

- ▶ Abstract types can be used to model enumerations
- ▶ Hierarchical modeling allows to reuse functionality

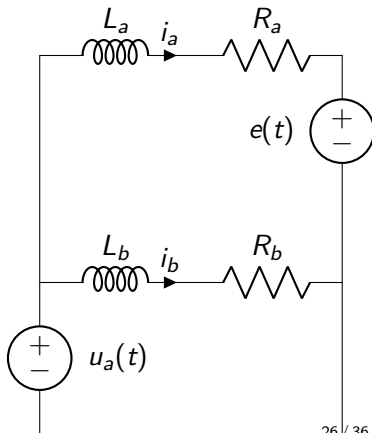
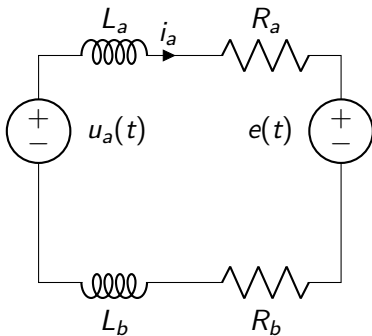
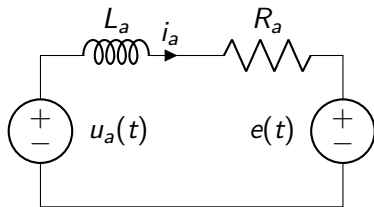
```
abstract type AbstractExcitation{T} end
struct Series{T} <: AbstractExcitation{T}
    winding::ExcitationWinding{T}
end
struct Parallel{T} <: AbstractExcitation{T}
    winding::ExcitationWinding{T}
end
struct Independent{T} <: AbstractExcitation{T}
    winding::ExcitationWinding{T}
end

struct DCMotor{T, E<:AbstractExcitation{T}}
    anchor::Anchor{T}
    excitation::E
end
```



Define a common interface, and use multiple dispatch for implementation

```
derivative(motor::DCMotorWithWinding{<:Independent}, u_a, u_b, i_a, i_b, w)  
derivative(motor::DCMotorWithWinding{<:Series}, u_a, u_b, i_a, i_b, w)  
derivative(motor::DCMotorWithWinding{<:Parallel}, u_a, u_b, i_a, i_b, w)
```





Creational patterns for larger models I

Deserialisation becomes a larger task in and of itself

- ▶ Take a hierarchical format to read the config from
- ▶ Use a context variable to keep track of *non-local* values
- ▶ Create a dedicated deserialise function to allow future new description formats (don't use constructor functions)

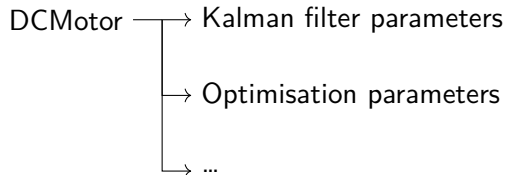
```
function deserialise(cfg::Dict, target::Type{T}, context::Dict
) where {T <: MotorWithLoad}
  context[:U_pu] = cfg[:pu][:U]
  context[:S_pu] = cfg[:pu][:S]
  motor = deserialise(cfg[:motor], AnyMotor, context)
  # ...
end
```



Creational patterns for larger models II

```
function deserialise(cfg::Dict, target::Type{<:AnyMotor}, context::Dict)
    if cfg[:type] == "DC"
        deserialise(cfg, DCMotor, context)
    elseif cfg[:type] == "AC"
        deserialise(cfg, ACMotor, context)
    else
        throw(ValueError("Unknown motor type $(cfg[:type])"))
    end
end
```

```
motor:
  type: "DC"
  U_a_nom: 400
  P_nom: 1000
  kalmanfilter:
    R1: [0.1]
  optimisation:
    alpha: 0.5
```





Handling multiple modeling options

- Chosen model depends on context

```
function deserialise(cfg::Dict, target::Type{<:DCMotor}, context)
  if context[:DCMotorContext] == "linear"
    deserialise(cfg, LinearDCMotor, context)
  elseif context[:DCMotorContext] == "nonlinear"
    deserialise(cfg, NonlinearDCMotor, context)
  else
    throw(ValueError("Unknown DCMotor context"))
  end
end
```

System modeling is our core business and it's hard

- ▶ Be robust against modeling mistakes (i.e. garbage in, error out)
- ▶ Different physical/digital components to model
- ▶ Different ways to model the same components
- ▶ **Different ways to compose components into a larger system**
- ▶ **Different tasks to do (simulation, optimisation, state estimation)**

Many submodels together create one solver-friendly model

- ▶ For complex models, the **Builder pattern** is often useful
- ▶ Standardise interfaces, or create helper functions to allow easily swapping components

```
m = JuMP.model(Ipopt.Optimizer)
add_model!(m, my_motor) # motor equations and variables
add_model!(m, my_load)  # load equations and variables
connect!(m, my_motor, my_load) # bare minimum to "connect" both equation sets
JuMP.optimize!(m)
```

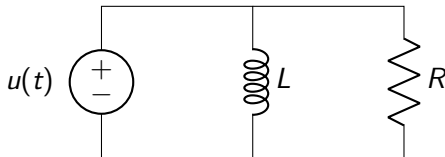
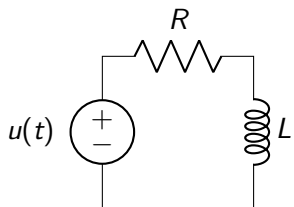
```
m = JuMP.model(Ipopt.Optimizer)
add_model!(m, my_other_motor) # maybe a completely different type of motor
add_model!(m, my_load)
connect!(m, my_other_motor, my_load) # dispatch to the correct implementation
JuMP.optimize!(m)
```



finished_product

400g of flour, 6 eggs and 1l of milk do not make pancakes

- ▶ It's impossible to completely automate composition of submodels/components
- ▶ Use a context to describe connections between components



```
simulate(Circuit([U(t -> 230), Parallel([R(1.0), L(1.0)])]))
```



finished_product

Composing components is not easy

- ▶ n components with m_n possible implementations → exponentially many possible combinations
- ▶ How are shared variables handled? Which component, if any, owns it?
- ▶ How to design the type hierarchy?
- ▶ Where to collect model information and how to ensure it remains valid?
- ▶ How to handle causality (changing input/output variables)?
- ▶ Aim for good enough, not for perfect

Design Patterns: building blocks for software

- ▶ Design patterns solve a specific problem
- ▶ Multiple design patterns can be combined into an application
- ▶ Sound use of design patterns makes for more reusable/extensible code

Further reference

- ▶ Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. 1st ed. Addison-Wesley Professional Computing Series. United States: Addison-Wesley Professional, Oct. 31, 1994. 395 pp. ISBN: 978-0-201-63361-0
- ▶ <https://refactoring.guru/design-patterns>

Want to improve your Julia skills?

- ▶ Take an old project and apply the concepts of today
- ▶ Contribute to your favorite Julia package
- ▶ Look at other people's code
 - ▶ Learn from both good and bad examples and recognise both
- ▶ ...

Thank you for attending

- ▶ Lucas Bex
 - ▶ `<cas.bex@kuleuven.be>` ELEC 00.93 row 2 column 1 facing south
- ▶ Matteo Rossini
 - ▶ `<matteo.rossini@kuleuven.be>` ELEC 00.93 row 3 column 2 facing north
- ▶ Jeroen Tant
 - ▶ `<jeroen.tant@kuleuven.be>` ELEC 00.08